# MegaMOO — Merchant Setup Guide

A **merchant** turns a room into a shop: players `buy` goods from a shopkeeper, `offer` coin to close the sale, and walk away with a freshly minted item. This guide is a practical, step-by-step reference for setting up and installing a merchant — using a **bar** (a shopkeeper who serves drinks) as the worked example, since it exercises every moving part: the price book, the shopkeeper's speech, a container item, and a liquid the container holds.

> **Scope.** This is *game content*, not part of the MegaMOO engine. It documents the merchant/commerce system built on top of the engine — the `buy` / `offer` verbs on **#17 ICRoom**, the **#92 BaseMerchant** price book, and the drink stack (**#28 CupContainer** / **#91 BaseDrinkable**). The engine itself is documented in the MegaMOO Developer Manual.



---

## How a merchant works

The key idea: **the merchant is a back-end price book, not an in-room NPC.** It has no location. A room points at it with a single `merchant` property — set it to `#N` and reading `room.merchant` gives back the merchant object. The shopkeeper the player "sees" is an entry in the merchant's `npcs` dictionary — a name and a set of spoken lines — never a walking, physical object.

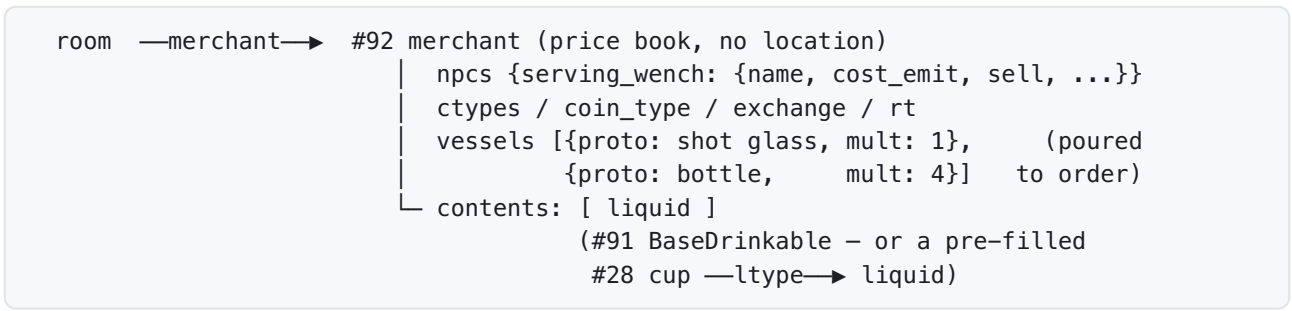
A purchase is a two-step conversation:

1. **`buy <item>`** — the room's `buy` verb (on **#17 ICRoom**) finds the merchant, matches the requested item against the merchant's stock, asks the merchant what it costs, and **quotes** a price. Nothing changes hands. The quote is stashed on the buyer as `pending_buy`.
2. **`offer`** — the `offer` verb reads `pending_buy` back, checks the coin, the amount, and the buyer's funds and free hands, then **mints** a copy of the stock item into the buyer's hand and subtracts the cost from their cash. A bare `offer` pays the asking price outright in the merchant's coin; `offer <amount> <coin>` states an explicit bid (below the asking price it's refused).

For a bar, the minted item is a real **#28 CupContainer** holding a **#91 BaseDrinkable** liquid, so the drink the player buys can actually be consumed with `drink` afterward. A bar can stock its drinks either of two ways:

- **Poured to order** (recommended, and what the live demo uses) — the stock item is the bare *liquid*, and the merchant carries a `vessels` list of empty cup/bottle prototypes it can serve that liquid in, each at its own price multiple. One stocked drink covers every serving size. See Poured to order.
- **Pre-filled cup stock** — the stock item is a filled cup prototype; every sale clones it as-is. Simplest when a drink comes in exactly one form.

Quantity is part of the quote, too: `buy case of <item>` quotes twelve at twelve times the unit price, `buy half a case of <item>` six — and `offer` delivers them boxed. See Buying by the case.



## The four objects

| Object | Prototype | Role |
|---|---|---|
| **The room** | #17 ICRoom (or a child) | Where the shop is. Points at the merchant via `merchant`. |
| **The merchant** | child of #92 BaseMerchant | The price book + shopkeeper speech. No location. |
| **The liquid** | child of #91 BaseDrinkable | The drink itself — for a poured-to-order bar this IS the stock item. Supplies drink messages and effects. |
| **The vessels** | children of #28 CupContainer | Empty cup/bottle prototypes listed in `merchant.vessels` (poured to order) — or a single filled cup used directly as stock (pre-filled style). Cloned on each sale. |

A merchant that sells a non-drink item needs just the room, the merchant, and a stock item (any sellable object); the liquid/vessel pairing is specific to bars and other drink vendors.

---

## Installation, step by step

Staff build merchants with the in-game `@` **builder commands** (`@make`, `@set`, `@adprop`); AI helpers can do the same, or use the equivalent `set_property` / `search_objects` MCP tools. Both write through the same engine path, so the two are interchangeable. The examples below use `@` commands.

`@make <parent> = <name>` creates a child of `<parent>` (the `<name>` is the new object's noun) and **reports the new object's number** — the examples below use the demo fixture's numbers (#5029, #5026, #5025) so you can follow along.

**Object names are composed, not set as one string.** A displayed title is built from the `name_mod_list` — `[article, adj1, adj2, adj3, trailer]` — plus the `noun`. You build it with the naming verbs rather than `@set …name`:

| Verb | Sets | Example |
|---|---|---|
| `@name <o> = <noun>` | the noun (+ auto `a` / `an`) | `@name #5032 = glass` → "a glass" |
| `@article <o> = <art>` | slot 0 (must be in `GOOD_ARTICLES`) | `@article #5032 = a` |
| `@adjective <o> = <a1> [a2] [a3]` | slots 1–3 | `@adjective #5032 = shot` → "a shot glass" |
| `@trailer <o> = <text>` | slot 4 (after the noun) | `@trailer #5030 = , still steaming` |

(`@art` / `@adj` abbreviations work.) Article + adjectives + noun compose the title, e.g. `a` + `shot` + `glass` → **"a shot glass"**.

**A cup's title is the vessel alone — the liquid never appears in it.** A shot glass holding fire whiskey is titled "a shot glass", full or empty; what it holds shows on `look in glass`, not in the name. The `_ctitle` verb on #28 enforces this (it clears any liquid trailer a legacy cup may carry and regenerates the name), so don't hand-set a cup's trailer. Matching follows the title: players handle the cup by its vessel words — `drink glass`, `drink shot glass`, or any pmatch prefix (`drink sh gl`) — and deliberately NOT by the liquid (`drink whiskey` is a miss). The `buy` verb still resolves liquid words against *stock* by checking what each cup holds (its `ltype`) and, poured to order, by matching the stocked liquid itself.

The steps below build the poured-to-order bar (the live demo). For the pre-filled alternative, see the note after step 4.

1. **Create the liquid** as a child of **#91 BaseDrinkable** — "fire whiskey" is the adjective `fire` + noun `whiskey`, with no article. Give it a `value`: the unit price every serving is computed from:

   ```
   @make #91 = whiskey
   @adjective #5029 = fire
   @article #5029 =          (clear the article -> "fire whiskey")
   @set #5029.value = 57
   ```

   Every naming verb regenerates `name` *and* `cname` (the capitalized form `%D` renders) — never `@set` those two directly. If an object was named by raw property writes instead (MCP `set_property`, eval), `@title <object>` rebuilds them from the noun and `name_mod_list`.

2. **Create the vessel prototypes** as children of **#28 CupContainer** — one per serving size, **empty** (no `ltype`; the vessel's name is just the vessel). Set `uses`, the capacity in drinks, then park them out of the world in **#6 Nowhere**. They must NOT sit in the merchant's stock, or the empties themselves would be buyable:

```
@make #28 = glass
@adjective #5032 = shot   ("a shot glass")
@set #5032.uses = 3
@move #5032 to #6

@make #28 = bottle        ("a bottle")
@set #5033.uses = 12
@move #5033 to #6
```

3. **Create the merchant** as a child of **#92 BaseMerchant**. Set `npcs` (the shopkeeper's name and lines), the coin model (`coin_type`, `exchange`), optional `rt` — and `vessels`, the serving options: each entry pairs a vessel prototype with a price multiplier, and the **first entry is the default serving**. `ctypes` is inherited from #92 and shared by all merchants — don't set it per-merchant. The `name` is just an internal identifier.

```
@make #92 = bar_merchant
@set #5025.npcs = {'serving_wench': {'name': 'a wrinkled old serving wench', ...}}
@set #5025.vessels = [{'proto': 5032, 'mult': 1}, {'proto': 5033, 'mult': 4}]
```

4. **Stock the merchant.** Put the *liquid* in the merchant's `contents` (the default stock location) — `@move #5029 to #5025` — or list stock holders explicitly in `merchant.items` (a list of references — bare `#N` works here too: `@set #5025.items = [#5030]`).

> **Pre-filled cup stock (the alternative).** Skip `vessels` and stock a filled cup instead: make one #28 child, set `value`, `uses`, `cuses` (current fill), and `ltype = #5029`. Its title stays the bare vessel ("a shot glass") — `buy whiskey` still finds it because `buy` checks each stocked cup's `ltype`, and the quote composes the description ("a shot glass of fire whiskey costs 57 plats"). Every sale clones it as-is. The retired demo cup **#5026** in #6 Nowhere is a working example.

5. **Wire the room to the merchant** — again a bare `#N` reference:

```
@set #401.merchant = #5025
```

6. **Prepare buyers.** Confirm characters have a `cash` list and a `pending_buy` property (both established by `@initchar`; legacy characters may need a backfill).

7. **Test.** `buy <item>`, then `offer` (or `offer <amount> <coin>`), then `drink <vessel>`.

The `@` commands and the `set_property` MCP tool both bypass the non-wizard eval gate and write through the same store path; the auto-reload watcher hot-loads any disk verb edits.

---

## Poured to order: one liquid, many vessels

Stock one drink, sell it in any serving size. The merchant stocks the **bare liquid** (a #91 BaseDrinkable child) and carries a `vessels` list:

```
vessels = [{'proto': 5032, 'mult': 1},    # a shot glass (uses 3)  — the default
           {'proto': 5033, 'mult': 4}]    # a bottle     (uses 12) — 4x the price
```

Each entry pairs an **empty #28 vessel prototype** (parked in #6 Nowhere, not in stock) with a **price multiplier**. The first entry is the default serving. `vessels` is defined `[]` on #92, so a merchant without it simply has no serving options — and a stocked bare liquid with no vessels is refused with *"I've got nothing to serve that in."*

What the player can type:

| Command | Resolves to |
|---|---|
| `buy whiskey` | the liquid in the **default** vessel — "a shot glass of fire whiskey costs 57 plats" |
| `buy bottle of fire whiskey` | the liquid in the named vessel — 57 × 4 = "228 plats" |
| `buy glass of ale` | refused — the vessel matches but the liquid doesn't |

The quote prices the serving at `liquid.value × mult × exchange[coin_type]` and stashes the chosen vessel in `pending_buy` (`vproto` / `vmult`). On a successful `offer`, the mint clones the **vessel prototype** (not the liquid), then pours: `ltype` = the liquid, `cuses` = the vessel's `uses` (served full), `value` = what was charged. The full order description ("a bottle of fire whiskey") lives in the *quote*; the handover and the object itself use the bare vessel — "…serves you a bottle" — and `look in bottle` shows the fire whiskey, `drink bottle` works for twelve sips.

Pre-filled cup stock is unaffected: when `pending_buy` carries no vessel, the mint clones the stock item as-is.

**The two styles mix freely in one stock.** A merchant can hold filled cups and bare liquids side by side (with `vessels` serving the latter); each request resolves independently. Order of resolution when phrases overlap: a flat name match first (a bare liquid's name catches "buy whiskey"), then "vessel of liquid" against *filled cups* (their `ltype`), then bare liquid words against filled cups, and only then the poured-to-order vessel list — so if the same drink exists both ways, "buy whiskey" pours it to order while "buy glass of fire whiskey" sells the pre-filled cup.

---

## Buying by the case

Bulk quantity is parsed by `buy` before anything else, works with **both** stocking styles, and survives into `offer`:

| Command | Quantity | Quote |
|---|---|---|
| `buy case of fire whiskey` | 12 | “a case of fire whiskey costs 684 plats” (12 × 57) |
| `buy half a case of fire whiskey` | 6 | “a half-case of fire whiskey costs 342 plats” |
| `buy case of bottle of fire whiskey` | 12 | 12 × (57 × 4) = “2736 plats” — quantity and vessel compose |

`offer` honors the whole quote: it charges **quantity × unit cost** (an offer below the case price is refused with `offer_fail`), mints all the units — cloned stock items or poured vessels alike — and delivers them **boxed**. The box is a fresh child of **#26 BaseContainer**, titled bare like a cup — “a case” / “a half-case” (aliases `case` / `box`), never the contents — sized to its cargo, and priced at the full charge; each unit inside carries the per-unit price. Because the goods arrive as one box, **one free hand** is all the buyer needs — and afterward the box behaves like any container:

```
> look in case
In a case you see a shot glass, a shot glass, ... (x12)
> get glass from case
You get a shot glass from a case.
```

To keep something off the bulk menu, list it in the merchant’s `case_exclude` — a list of stock objnums (the item itself, or the liquid a filled cup holds). A case request for anything listed is refused with *“That’s not something I’ll sell by the case.”*

> **Matcher note.** The full order wording (“a case of fire whiskey”) exists only in the quote. The box, like the cups inside it, matches by its container words — `get case` / `get box` — and nothing in your hands answers to the liquid: `drink fire whiskey` is simply a miss. Drink from a glass (`drink glass`), drawing one out of the box first if need be.

---

## Property reference

### The room

| Property | Example | Notes |
|---|---|---|
| `merchant` | `#5025` | Object reference to the merchant (`@set here.merchant = #5025`). Reads back as the object. This one line is all the room needs. |

## The merchant (price book)

Child of **#92 BaseMerchant**. No location required.

| Property | Example | Notes |
|---|---|---|
| `name` | `"haven_bar_pricebook"` | Internal identifier only — never shown to players. |
| `npcs` | (dict, see below) | The shopkeeper(s): name + spoken lines. |
| `active_npc` | `"serving_wench"` | Optional. Which npc speaks. Omit → the first npc in `npcs`. |
| `ctypes` | `['plat']` | Coin names this merchant deals in, stored **singular**. (#92 default `['plat']`.) |
| `coin_type` | `0` | Index into `ctypes` — the coin used for this shop. |
| `exchange` | `[1]` | Multipliers parallel to `ctypes`. Cost = `item.value * exchange[coin_type]`. |
| `rt` | `3` | Haggle round-time in seconds after a quote. `0` or unset = no cooldown. |
| `items` | `[#5030]` | Optional list of stock-holder references. Omit → the merchant's own `contents` is the stock. |
| `vessels` | `[{'proto': 5032, 'mult': 1}, …]` | Serving options for poured-to-order liquids: empty #28 prototype + price multiplier, first entry = default. (#92 default `[]`.) |
| `case_exclude` | `[5029]` | Stock objnums (the item, or the liquid a filled cup holds) that can't be bought by the case/half-case. |
| `cost_help` | `"…make an 'offer'."` | Optional merchant-wide override of the how-to-buy prompt appended to every quote line (npc `cost_help` wins; `''` suppresses). |

**The coin word is derived, not stored separately.** There is no `currency` property — the word the player sees is `ctypes[coin_type]`. Store it singular (`'plat'`, not `'plats'`): the verbs append an `'s'` for display whenever the amount isn't exactly 1 (`"57 plats"`, `"1 plat"`), and `offer` matches a typed coin with `startswith`, so a singular `'plat'` matches both `plat` and `plats` typed by the player — a plural entry would only match `plats`.

The `npcs` dictionary maps an npc key to a sub-dictionary of spoken lines:

```
{
    'serving_wench': {
      'name':      "a wrinkled old serving wench",
      'cost_emit': "Lessee eer, %d costs %1 %2. Ye want one?",
      'sell':      "'Aye, cummin oop,' and serves you %d.",
      'o_sell':    "'Aye, cummin oop,' and serves %d %i.",
      # Optional refusals — #92 supplies defaults for all three:
      # 'coin_fail':  "We only take %1 here. Sorry.",
      # 'offer_fail': "%D costs %1. Sorry.",
      # 'cash_fail':  "Ye don't have that kind o' coin.",
      # Optional how-to-buy prompt, appended after the spoken quote
      # ('' suppresses; omit for the default):
      # 'cost_help':  "Just say 'offer %1 %2' if ye fancy it.",
    }
}
```

| npc field | Example | Rendered when… |
|---|---|---|
| `name` | `"a wrinkled old serving wench"` | Always. This is what the player hears. `buy` / `offer` wrap it as `Name says, "…"`. |
| `cost_emit` | `"Lessee eer, %d costs %1 %2. Ye want one?"` | On a quote. `%d` = item, `%1` = price, `%2` = currency. Speech only — the how-to-buy instruction ( `cost_help` ) is appended after it. |
| `cost_help` | `"If you would like to purchase %d make an 'offer'."` | Appended to the quote line **after the closing quote mark**, unquoted (a house prompt, not shopkeeper speech). That string is the built-in default; override per npc or per merchant ( `merchant.cost_help` ), or set `''` to suppress. |
| `sell` | `"'Aye, cummin oop,' and serves you %d."` | To the buyer on a closed sale. Carries its own quote marks (emitted raw). `%d` = the goods. |
| `o_sell` | `"'Aye, cummin oop,' and serves %d %i."` | To the room on a closed sale. `%d` = the buyer, `%i` = the goods. |
| `coin_fail` | `"We only take %1 here. Sorry."` | Wrong coin offered. `%1` = the accepted coin. Optional (#92 default). |
| `offer_fail` | `"%D costs %1. Sorry."` | Offer too low. `%D` = item (capitalized `cname` ), `%1` = cost phrase. Optional. |
| `cash_fail` | `"Ye don't have that kind o' coin."` | Buyer lacks funds. Optional (#92 default). |

**Token convention.** Objects render through dob / iob ( `%d` / `%D` , `%i` / `%I` ); raw strings (price numbers, coin words) render through numbered slots ( `%1` , `%2` , …). Never route an object's name through a numbered slot — use `%d` . `sell` and `o_sell` carry their own quotation marks, so they are emitted raw rather than wrapped in the `Name says, "…"` form.

Because a cup's title omits its liquid (and a case/poured order has no object at all at quote time), `buy` splices the quote's composed description ("a shot glass of fire whiskey", "a case of fire whiskey") over the goods tokens in `cost_emit` and `cost_help` whenever it says more than the object's own name. The *handover* is the opposite: `sell` / `o_sell` render the minted goods by their own name — the quote describes the order, the serve line just hands over "a shot glass". No template changes are needed.

## Cups and vessels

Children of **#28 CupContainer** — either an *empty vessel prototype* listed in `merchant.vessels`, or a *pre-filled cup* used directly as stock. Both are prototypes that get cloned on every sale; the buyer receives a copy.

A cup's name is **structured and vessel-only**: noun (`glass`) + `name_mod_list` adjectives (`shot`) title the vessel — "a shot glass" — and that's the whole title, full or empty. The liquid never appears in it; what the cup holds shows on `look in glass`. Don't hand-set `name`, `cname`, or the trailer on a cup; compose the vessel with `@name` / `@adjective` and let `_title` generate the rest (`_ctitle` strips any liquid trailer left on a legacy cup). Players match the cup by its vessel words (`drink shot glass`, `get glass`, pmatch prefixes) — not by the liquid.

| Property | Example | Notes |
|---|---|---|
| `noun` | `"glass"` | Native attribute (not a MOO property). The vessel word the matcher keys on. |
| `aliases` | `["glass", "cup"]` | Native attribute. Extra match words. |
| `uses` | `3` | Capacity in drinks (defined on #28). `look in` reports fullness from `cuses` / `uses`. |
| `cuses` | `3` | Current fill — drinks left before the cup empties. A poured serving is minted full (`cuses` = the vessel's `uses`). |
| `ltype` | `#5029` | Object reference to the liquid held. Set on a pre-filled stock cup; set by `offer` when pouring to order. Empty vessel prototypes leave it unset. |
| `open` | `True` | The cup must be open, or `drink` / `cdrink` refuses it. |
| `value` | `57` | Pre-filled stock: base price before the exchange multiplier. Poured to order: leave unset — the minted copy is priced at what was charged. |
| `hands` | `1` | Hands needed to hold it (1 or 2). Defaults to 1 if unset. |
| `weight` | `1` | Added to the buyer's carried load. |

An **empty vessel prototype** needs only its composed name, `uses`, and a home in #6 Nowhere. A **pre-filled stock cup** additionally sets `value`, `cuses`, and `ltype` — its title is unchanged by the fill.

### The liquid

Child of **#91 BaseDrinkable**. Beyond `name` and `value`, everything is optional — the `cdrink` verb has a sensible fallback for each. The live **#5029** sets `name` and `value` (`cname` comes along free from `_title`).

| Property | Example | Notes |
|---|---|---|
| `name` | `"fire whiskey"` | Composed adjective + noun (no article). Shown in drink lines and in every generated cup/box name. |
| `cname` | `"Fire whiskey"` | Capitalized name — what `%D` renders (e.g. in `offer_fail`). Set automatically by `_title` whenever the name is composed with the naming verbs; don't hand-set it. |
| `value` | `57` | The unit price. Poured to order prices every serving from it (`value × mult`); `_cost` also falls back to it when a filled stock cup has no value of its own. |
| `ceemits` | (see below) | Four message pools: `[player_prep, room_prep, player_raw, room_raw]`. |
| `prepared` | `True` | Picks the prep pair vs. the raw pair from `ceemits`. Default `False`. |
| `effects` | `[('drunk', 2, 60)]` | Effect tuples passed to `eu.trigger_all`. |
| `effect_chance` | `75` | Percent chance effects apply. Default `100`. |
| `effects_per_bite` | `False` | `False` = effects only on the last drink. Default `True`. |
| `rt_dice` | `[1, 7, 0]` | Round-time dice `[num, sides, offset]`. Default `[1, 7, 0]`. |
| `finish` | `"You drain the last of %d from %i."` | Shown to the drinker on the final sip. |
| `ofinish` | `"%S drains the last of %d from %i."` | Shown to the room on the final sip. |

`ceemits` is a list of four message pools; the drinker randomly gets one line from the relevant pool. `prepared` chooses between the "prepared" pair (indices 0–1) and the "raw" pair (indices 2–3):

```
ceemits = [
  ["You sip %d from %i."],           # player, prepared
  ["%S sips %d from %i."],           # room,   prepared
  ["You gulp raw %d from %i."],      # player, raw
  ["%S gulps raw %d from %i."],      # room,   raw
]
```

In drink messages, `%d` = the liquid, `%i` = the cup, `%S` = the drinker.

---

## Buyer prerequisites

A character has to be able to pay and to receive goods:

| Property | Example | Notes |
|---|---|---|
| `cash` | `[100]` | A **list indexed by `coin_type`** — `[100]` is 100 plats. |
| `pending_buy` | `{}` | Scratch dict where `buy` stashes the quote for `offer`. Declared by `@initchar`. |

Both are established when a character is initialized with `@initchar`. Characters created before these defaults existed need a one-time backfill — a player-context verb can *write* an existing property but cannot *create* a new one on a staff-owned character, so the property must be added first.

---

## A complete worked example

The live demo in room **#401** (“Haven — East Main St”) is a poured-to-order bar:

| Object | # | Setup |
|---|---|---|
| Room | #401 | `merchant = "#5025"` |
| Merchant | #5025 | `npcs` = serving_wench; `coin_type=0`, `exchange=[1]`, `rt=3`; `vessels = [{'proto': 5032, 'mult': 1}, {'proto': 5033, 'mult': 4}]`; stock (contents) = the liquid #5029 |
| Liquid (stock) | #5029 | `name="fire whiskey"` (adjective `fire` + noun `whiskey`), `value=57` |
| Vessel: shot glass | #5032 | “a shot glass”, `uses=3`, empty — parked in #6 Nowhere |
| Vessel: bottle | #5033 | “a bottle”, `uses=12`, empty — parked in #6 Nowhere |
| Buyer | #5021 | `cash=[…]`, indexed by `coin_type` |

(The retired pre-filled cup **#5026** also sits in #6 Nowhere as a working example of the pre-filled stocking style.)

The sale as a player sees it — default vessel, explicit vessel, and a case:

```
> buy shot glass of whis
A wrinkled old serving wench says, "Lessee eer, a shot glass of fire whiskey
costs 57 plats. Ye want one?" If you would like to purchase a shot glass of
fire whiskey make an 'offer'.

> offer
'Aye, cummin oop,' and serves you a shot glass.

> inventory
You have a shot glass in your right hand.

> drink glass
You take a drink of fire whiskey from a shot glass.

> buy bottle of fire whiskey
A wrinkled old serving wench says, "Lessee eer, a bottle of fire whiskey
costs 228 plats. Ye want one?" If you would like to purchase a bottle of
fire whiskey make an 'offer'.

> buy case of fire whiskey
A wrinkled old serving wench says, "Lessee eer, a case of fire whiskey
costs 684 plats. Ye want one?" If you would like to purchase a case of
fire whiskey make an 'offer'.

> offer 684 plats
'Aye, cummin oop,' and serves you a case.

> get glass from case
You get a shot glass from a case.
```

(`buy` phrases abbreviate — `buy shot glass of whis` matches by pmatch prefixes. `offer` alone pays the asking price; `offer 684 plats` states the bid explicitly.)

Every minted drink is a real #28 cup poured full (`cuses` = its vessel's `uses`), so the shot glass drinks three times and the bottle twelve before they empty. The *quote* names the full order ("a shot glass of fire whiskey"); the serve line and the glass itself use the bare vessel ("a shot glass") throughout — `look in glass` is how you see what's in it.

---

## Testing your merchant

Run through the full path and confirm each stage:

1. **Quote** — `buy <word>` using each of the item's `noun` / `aliases`. The shopkeeper should quote the price with the coin word from `ctypes[coin_type]`, pluralized to match the amount (`"57 plats"`, `"1 plat"`), with the how-to-buy prompt appended after the closing quote mark ("If you would like to purchase … make an 'offer'." — or the npc/merchant `cost_help` override).
2. **Wrong coin** — `offer 57 gold` (a coin the merchant doesn't take) should trigger `coin_fail`.

3. **Low offer** — `offer 1 plats` should trigger `offer_fail`.
4. **No funds** — a buyer with empty `cash` should trigger `cash_fail` (a bare `offer` too — accepting the quote still checks funds).
5. **Success** — a bare `offer` (and equally `offer <cost> <coin>`) should mint the item into the buyer's hand, subtract the cost, and emit `sell` (to the buyer) + `o_sell` (to the room), naming the goods by their own title ("serves you a shot glass").
6. **Consume** — for a bar, `drink <vessel>` (`drink glass`, `drink shot glass`, prefixes) should work and decrement `cuses`; `drink <liquid>` (`drink whiskey`) should NOT match — the liquid is not in the cup's name or match words, by design. The title stays "a shot glass" when drunk dry; `look in glass` reports the fill.
7. **Vessels** (poured to order) — `buy <liquid>` should quote the *default* vessel; `buy <vessel> of <liquid>` each other entry at its multiplier; a wrong pairing (`buy glass of ale`) should be refused. The *quote* should name the full order ("a shot glass of fire whiskey") while the minted drink and the serve line use the bare vessel.
8. **Cases** — `buy case of <item>` should quote 12 × the unit price (6 for `half a case`), an offer below that should trigger `offer_fail`, and a full offer should deliver **one box** holding all the units (`look in case`, `get <item> from case`). Anything in `case_exclude` should be refused at quote time.

---

# Troubleshooting

| Symptom | Likely cause |
| --- | --- |
| “There is no one to buy from here.” | `room.merchant` unset, or points at a bad `"#N"`. |
| “I don’t have any of that.” | Stock item not in the merchant’s `contents` (or `items`), or the player’s word doesn’t match the item’s `noun` / `aliases`. |
| Quote shows a blank/odd name | Object routed through a numbered slot instead of `%d`; or a stale/missing `cname` on a `%D` line — the object was named by raw property writes. `@title <object>` rebuilds name and cname (the naming verbs do this automatically). |
| Shopkeeper says nothing on sale | `sell` / `o_sell` unset on the npc, or the buyer is `invis` (suppresses `o_sell`). |
| “%D is closed.” on drink | Cup `open` is not `True`. |
| “%D is empty.” on drink | Cup has no `ltype`, or `cuses` reached zero. |
| Wrong price | Check `value` on the item (or its liquid) and `exchange[coin_type]` on the merchant — unit cost is their product, times the vessel `mult` and the case quantity. |
| Coin word doubled (“platss”) | `ctypes` holds the **plural** form; store it singular (`'plat'`) and let the verb append the `'s'`. |
| Player types “plat” but it isn’t accepted | `ctypes` holds the plural; store singular so `offer`’s `startswith` match accepts both `plat` and `plats`. |
| “I’ve got nothing to serve that in.” | A bare liquid is stocked but the merchant has no (resolvable) `vessels` entries. |
| `buy bottle of <liquid>` says “I don’t have any of that.” | The vessel isn’t in `vessels` (or its `proto` objnum is wrong), or the liquid isn’t in stock. Vessel words match the *prototype’s* noun/aliases. |
| Empty glasses/bottles are buyable | Vessel prototypes were left in the merchant’s `contents` / `items` — they belong in #6 Nowhere, referenced only from `vessels`. |
| “That’s not something I’ll sell by the case.” | The item (or its liquid) is listed in the merchant’s `case_exclude`. |
| Case quote is right but the wrong amount was charged | The buy and offer verbs disagree — both must be current; `offer` charges quantity × unit cost from `pending_buy`’s `qty`. |
| `drink whiskey` answers “Drink what?” | By design — cups and cases match by their container words only (`drink glass`, `get case`). The liquid is on `look`, not in any title. |
| A cup is still titled “… of fire whiskey” | |

| Symptom | Likely cause |
| --- | --- |
| | A legacy cup from before the vessel-only naming — the liquid is stuck in its trailer slot. `@trailer <cup>` (no text) clears the trailer and re-titles it. |